

CHAPTER IV

METHODOLOGY, RESULTS AND DISCUSSION

This chapter presents the methodology used in the development of the Egg Crack Detection System. The methodology is organized around the incremental model because the project was built as a series of working improvements instead of a single one-time release. Each increment added a usable part of the system, such as the desktop interface, backend service, image processing pipeline, camera workflow, batch processing, history storage, and report generation. After each increment, the output was reviewed, tested, and refined before the next set of features was added.

The system was implemented as a Flutter Windows desktop application connected to a Python FastAPI backend. Flutter handles the user interface, navigation, camera preview, file selection, and presentation of results. The backend handles image validation and crack detection using OpenCV-based processing. This separation allowed the development work to progress in clear increments: the frontend could be improved without changing the detection algorithm, while the backend could be tested independently through API endpoints and unit tests.

Research Design

The study used a developmental research design. The main output of the study is not only a written analysis but a functional software system that can accept egg images, detect possible cracks, display visual overlays, store accepted results, and export reports. The development process therefore focused on designing, building, testing, and improving a prototype until the major system functions were integrated into one desktop application.

The researchers selected an incremental software methodology because the project contains several dependent but separable modules. A working interface can be created first, then a backend service can be connected, then the detector can be improved, and finally camera, batch, history, and reporting functions can be added. This made the work more manageable and reduced the risk of waiting until the end of the project to discover integration problems.

Software Development Method: Incremental Model

The incremental model divides the system into smaller deliverable parts. Each part passes through requirement analysis, design, implementation, testing, deployment, and maintenance. The completed increment becomes part of the working system and serves as the base for the next increment. In this study, every increment produced a usable function that could be inspected and tested before additional features were introduced.

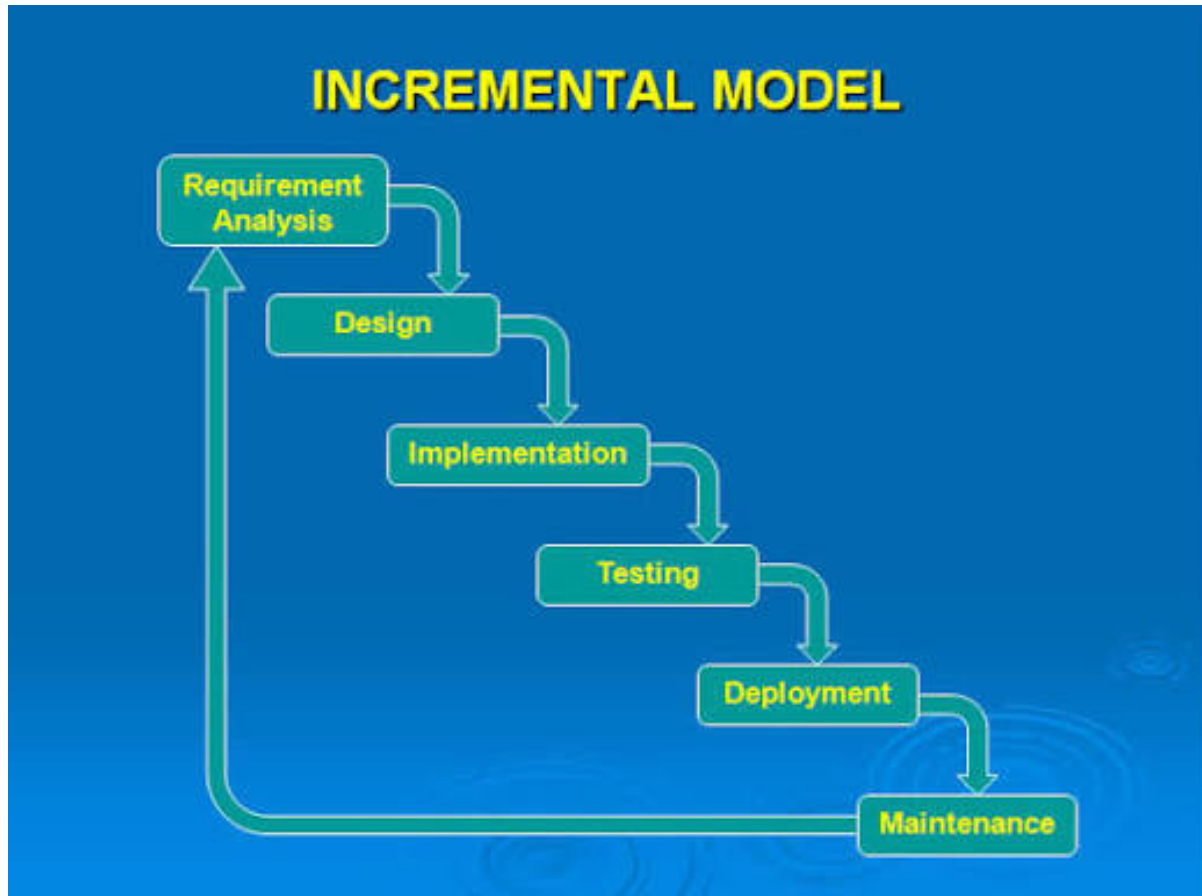


Figure 4.1. Incremental Model used as the development methodology.

The process began with a simple application shell and gradually expanded into a complete egg crack detection workflow. The first increment established the desktop application layout. The second increment connected the application to the backend service. The third increment implemented the crack detection logic. The succeeding increments added single-image detection, camera capture, focus assistance, batch processing, history, reporting, and packaging support. Each increment was treated as a partial release that had to run before the next increment was started.

Planning Phase

The planning phase defined the problem, the target users, the system boundary, and the development increments. The problem addressed by the system is the need for a desktop tool that can assist in identifying cracked eggs from captured or uploaded images. Manual checking can be time-consuming and subjective, especially when cracks are faint, thin, or affected by lighting. The system was planned to support image-based inspection by combining a clear desktop interface with a backend image-processing engine.

The project scope was limited to image-based egg crack detection using local desktop resources. The backend does not operate the camera directly; it only receives image bytes and returns detection results. The Flutter application is responsible for discovering cameras, showing the preview, capturing the image, sending it to the backend, and presenting the result. This scope was selected so that camera hardware responsibilities and image-processing responsibilities would remain separated and easier to maintain.

Table 4.1. Increment Plan for the Egg Crack Detection System

Increment	Main Focus	Development Output
1	Application foundation	Flutter desktop shell, routing, theme, dashboard, and dependency registration.
2	Backend and API integration	FastAPI server, health checking, automatic backend launcher, and Dio HTTP client.
3	Detection engine	OpenCV egg segmentation, image-quality checks, crack candidate extraction, fuzzy labeling, and overlay response.
4	Single-image workflow	Image picker, detection screen, Cubit state management, result model, and visual result display.
5	Camera workflow	Camera discovery, preview, focus scoring, capture and detect, orientation correction, and optional multi-frame validation.
6	Batch, history, and reports	Batch queue, local JSON history storage, deletion/clearing, summary metrics, CSV export, and deployment scripts.

Requirements Gathering and Analysis Phase

Requirements were gathered by identifying the tasks that an operator must complete when checking eggs through a desktop system. These tasks include selecting an existing image, using a camera to capture an egg, running detection, reading the result, saving accepted results, reviewing previous detections, processing several images, and exporting a report. The analysis also considered technical requirements such as local backend availability, supported

image formats, camera readiness, focus quality, and reliable handling of poor images.

Table 4.2. Functional and Non-Functional Requirements

Category	Requirement	Implementation Basis
Functional	The user can upload a single egg image for detection.	Implemented through the Flutter detection screen, file selector, detection repository, and backend /detect endpoint.
Functional	The user can capture an egg using a connected camera.	Implemented through CameraSessionService, CameraScreen, camera repository, and /detect/camera endpoint.
Functional	The system can show whether the egg is cracked or not cracked.	Implemented through DetectionResult fields such as is_crack, confidence, overlay_image_b64, and detection_score.
Functional	The system can process multiple selected images.	Implemented in the batch processing screen using a queue and repeated calls to the detection endpoint.
Functional	The system can store and export detection records.	Implemented through HistoryStore, /history endpoints, ReportsScreen, and /reports/export.
Non-Functional	The backend must be available before detection requests are sent.	Implemented through ApiService health polling and BackendLauncherService startup logic.
Non-Functional	Poor-quality images must not be confidently classified.	Implemented through image-quality metrics and HTTP 422 recapture behavior.
Non-Functional	The application should run locally on Windows desktop.	Implemented using Flutter desktop, local FastAPI server, backend scripts, and bundled backend startup support.

The analysis phase also identified expected data outputs. A detection result must contain a clear verdict, confidence value, processing time, original image, overlay image, egg size, crack size, quality metrics, candidate information, and optional intermediate processing images. These outputs are necessary because the user needs more than a yes-or-no answer; the system must also show why the result was produced and whether the capture was suitable.

Design Phase

The design phase converted the requirements into the structure of the application. The system was designed as two connected parts: a Flutter desktop frontend and a Python FastAPI backend. The frontend contains the screens, state management, camera service, file service, and API client. The backend contains upload validation, detection functions, history storage, and CSV export. The design follows separation of concerns so that the user interface, service communication, and image-processing algorithm can be improved independently.

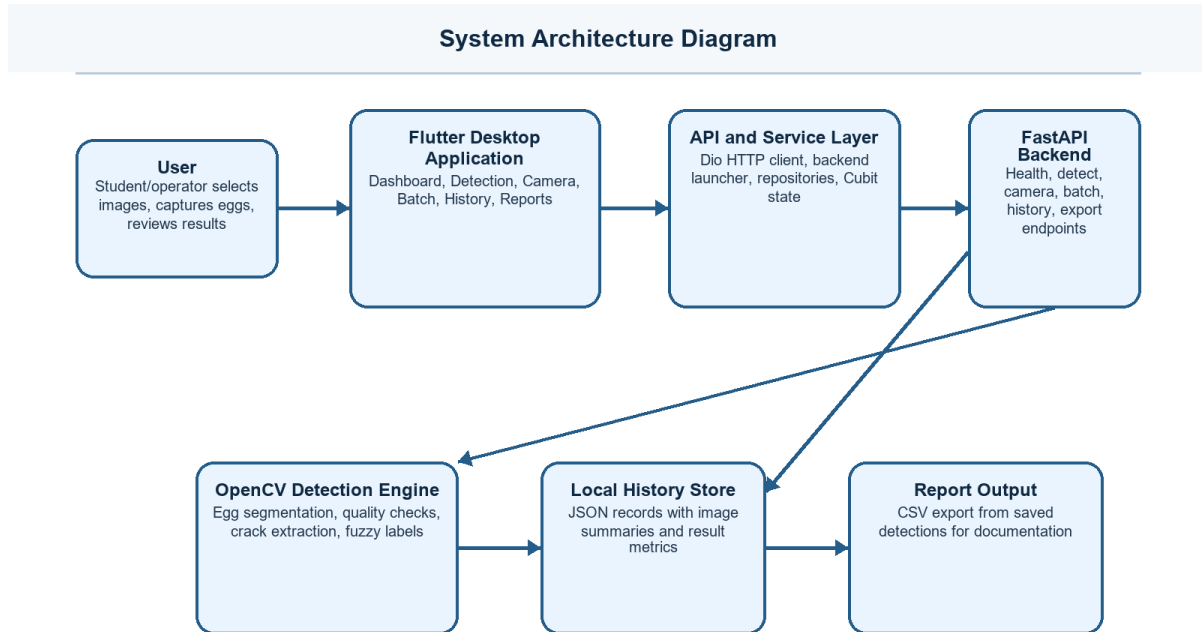


Figure 4.2. System architecture of the Egg Crack Detection System.

The architecture shows that the Flutter application is the main interaction layer. The user does not directly interact with the backend. Instead, the app sends requests through a Dio-based API service. Before any detection request is sent, the API service checks whether the backend is healthy and starts it when needed. The backend then processes the image and returns a JSON response containing both numeric metrics and images encoded in base64 format.

Use Case Diagram

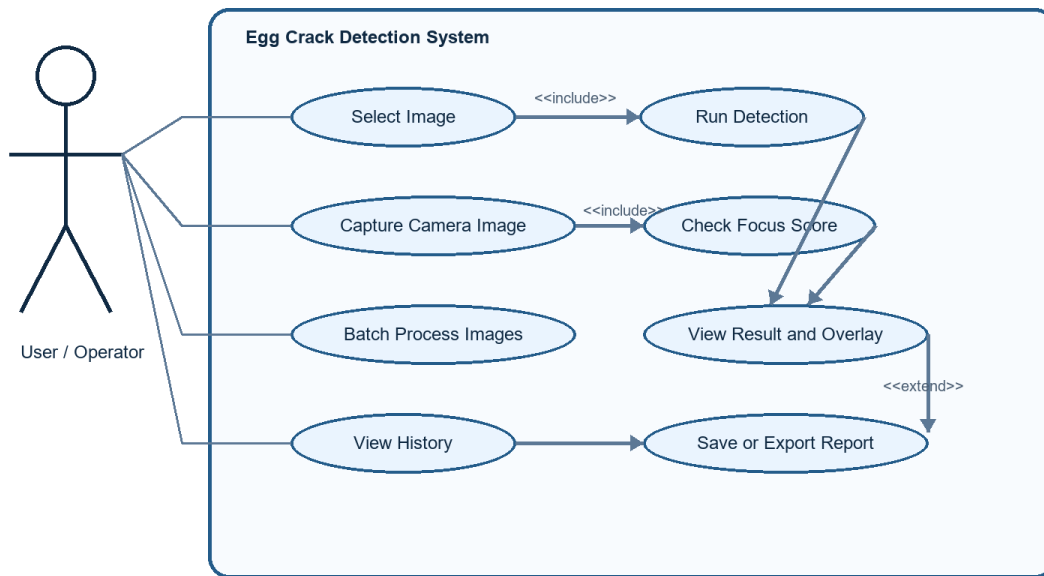


Figure 4.3. Use case diagram of the system.

The use case design centers on one primary actor: the user or operator. The operator can select an image, run detection, use the camera, review the result, save the result, process images in batch, view history, and export reports. These use cases guided the screen structure of the Flutter application. Each major use case became either a screen or a major screen action.

Activity Diagram for Detection Workflow

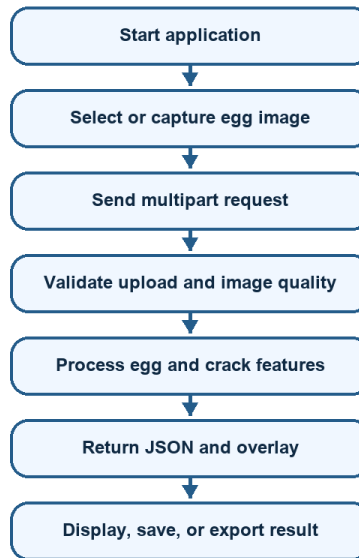


Figure 4.4. Activity diagram for the detection workflow.

Sequence Diagram for Single Image Detection

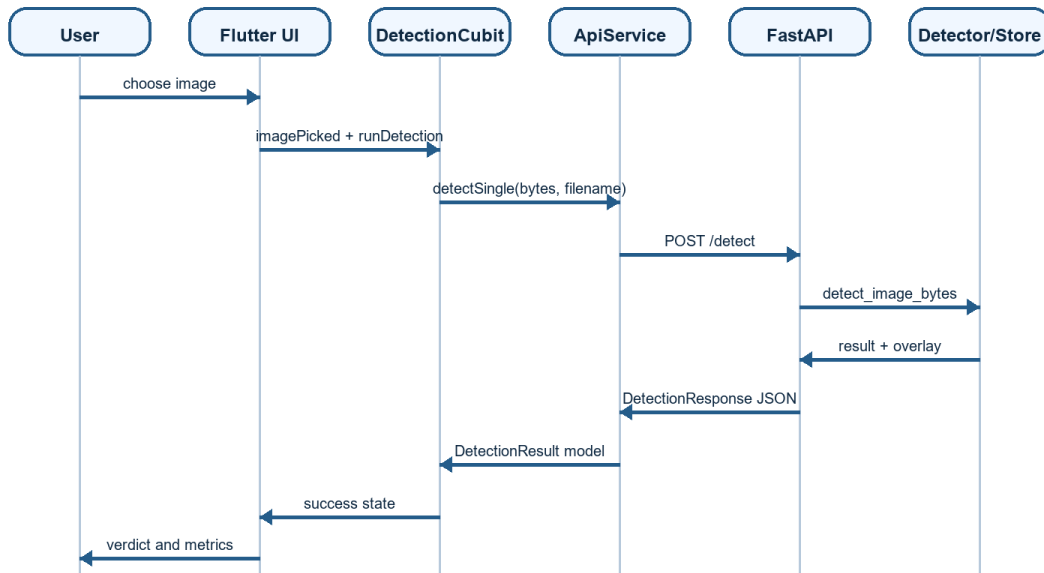


Figure 4.5. Sequence diagram for single-image detection.

The activity and sequence diagrams describe how a detection request moves through the system. When the user selects an image, the Flutter screen reads the file bytes and asks the Detection Cubit to run detection. The Cubit calls the repository, the repository uses ApiService, and ApiService sends a multipart request to the FastAPI backend. The backend validates the upload, runs the detector, optionally stores the result, and returns the response to the Flutter model.

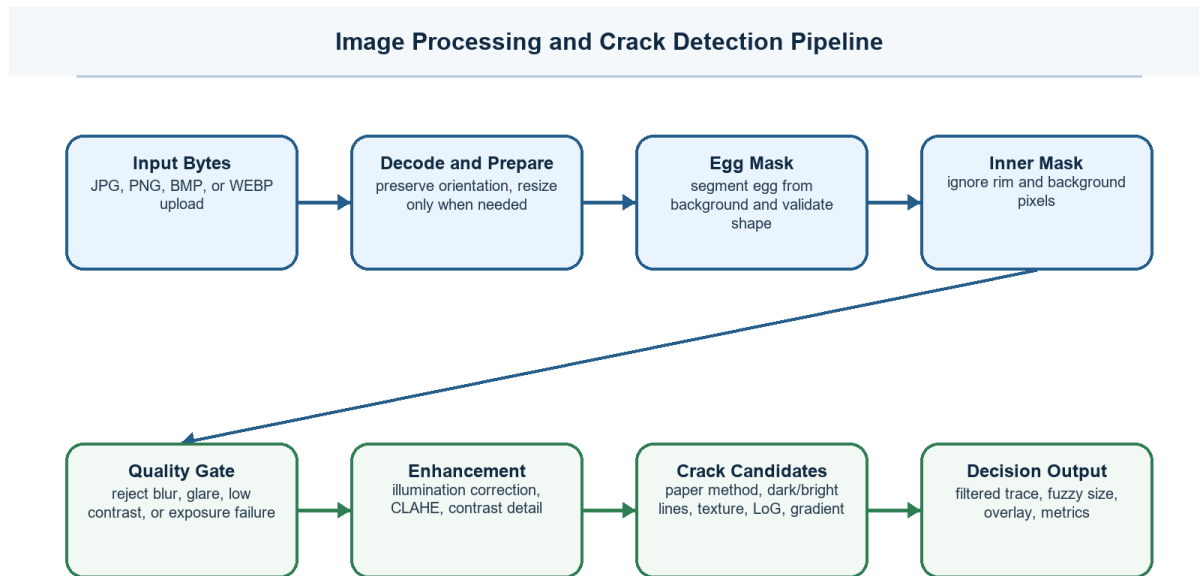


Figure 4.6. Image processing and crack detection pipeline.

The detection pipeline was designed to avoid analyzing the whole photo as if every pixel were part of the egg. The backend first detects the egg, builds an inner mask, and uses that mask to ignore the shell rim and the background. The image is then checked for quality problems such as blur, glare, underexposure, overexposure, and low contrast. Only suitable images proceed to the crack extraction steps. This design prevents the system from producing confident but unreliable results when the image itself is not usable.

History and Report Data Structure

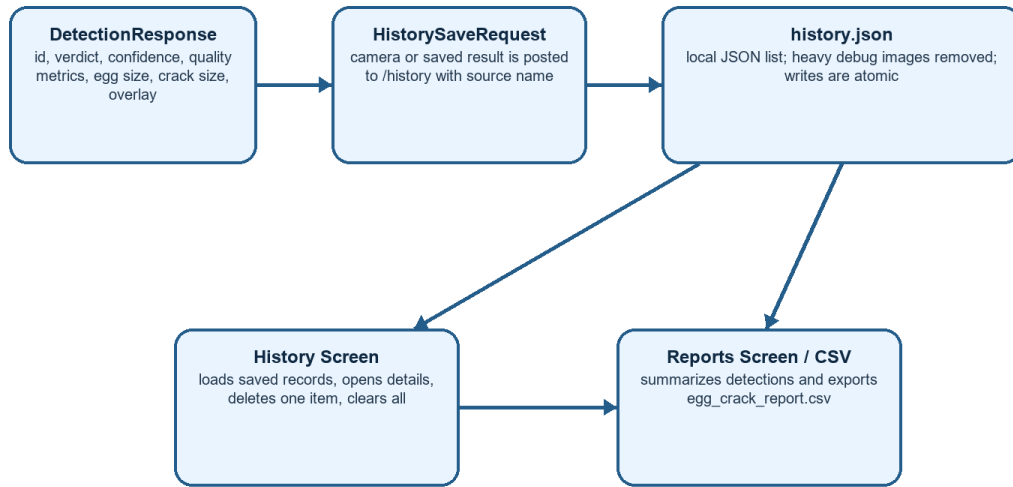


Figure 4.7. History and report data structure.

The system does not require an external database server. Detection records are stored locally in a JSON history file through the backend HistoryStore. The store removes heavy internal image fields before saving so the history file remains manageable. It also writes atomically so that a failed write does not corrupt the existing history file. The history screen retrieves saved records from the backend, while the reports screen uses the same data source to create a CSV export.

Development Phase

The development phase explains how the system was actually built. Following the incremental model, the researchers did not attempt to finish all modules at the same time. Instead, each increment added one working capability and then connected it to the previous work. This made the system easier to test because every increment produced visible output: a screen, an API response, a detection result, a camera capture, a saved record, or an exported report.

Increment 1: Flutter Desktop Foundation

The first increment established the desktop application foundation. The project was created as a Flutter desktop application for Windows. The main entry file initializes Flutter bindings, sets up the service locator, prepares the application window, and starts the app widget. The window configuration was important because the application is intended for inspection work, so it uses a desktop-sized layout rather than a small mobile-first layout.

Navigation was built with `go_router`. The router defines the main routes for the dashboard, single-image detection, camera detection, batch processing, history, and reports. These routes are placed inside a shared shell so that the application has a consistent navigation structure. The shared shell provides the frame of the application, while each feature screen provides its own content and actions. This design made it possible to add later increments without rewriting the whole interface.

The foundation increment also introduced project-wide services and dependency injection using `GetIt`. Services such as `BackendLauncherService`, `ApiService`, `FileService`, `CameraSessionService`, `DetectionRepository`, `CameraRepository`, and `DetectionCubit` are registered in one place. This means screens do not need to manually create their dependencies. Instead, they request the needed object from the service locator, making the application easier to extend and test.

Increment 2: Backend Service and API Connection

The second increment added the backend service and connected it to the Flutter frontend. The backend was built with `FastAPI` because it can expose lightweight HTTP endpoints and return structured JSON responses. The backend application defines endpoints for health checking, single-image detection, batch detection, camera detection, multi-frame camera detection, focus scoring, history operations, and CSV report export.

To make the desktop application easier to use, the frontend was designed to start or locate the backend automatically. BackendLauncherService searches for the backend executable or Python entry point and starts it when the backend is not already running. ApiService performs a health check before normal requests and waits until the backend is ready. If the backend cannot start, the Flutter side produces a friendly error message instead of allowing the request to fail silently.

The API connection was implemented with Dio. Detection requests are sent as multipart form data because images must be uploaded as binary file content. The detection repository sends a file name, the image bytes, and flags such as whether intermediate processing steps should be included. The backend validates the file extension, checks that the uploaded image is not empty, limits file size, and rejects unsupported inputs before the detector runs.

Increment 3: Image Processing and Detection Engine

The third increment developed the OpenCV-based detection engine. This was the most technical part of the system because the software had to identify a crack-like trace while avoiding false positives from the egg boundary, shell texture, glare, shadows, and camera noise. The backend receives the untouched image bytes, decodes them, preserves the captured orientation, and prepares a working image for analysis. Large images are resized only when needed so that processing remains efficient without immediately discarding useful camera detail.

The detector first builds candidate egg masks using thresholding, color information, and shape analysis. It selects the best egg contour by scoring whether the object is large enough, centered enough, and shaped like an egg rather than a random background object. After the egg mask is accepted, an inner egg mask is created using distance transform. The inner mask removes the shell rim and reduces the chance that the natural border of the egg will be mistaken for a crack.

Before crack extraction, the image passes through a quality gate. The detector computes values related to sharpness, local detail variance, saturated pixels, glare ratio, median brightness, and dynamic range. If the image is too blurry, too dark, overexposed, glare-heavy, or too low in contrast, the backend raises a recapture error instead of producing a questionable result. This step was added because detection accuracy depends strongly on the quality of the captured egg image.

For crack extraction, the detector combines several processing responses. It follows a paper-based crack detection procedure using red and green channel processing, Gaussian blur, egg masking, edge detection, morphology, and contour filtering. It also enhances local shell detail through illumination correction and CLAHE. Additional responses are generated for dark lines, bright lines, texture anomalies, Laplacian-of-Gaussian edges, gradient traces, dark valleys, and local hairline structures. These responses are combined because cracks can appear as dark lines, bright light leaks, or very faint traces depending on the illumination.

After candidate pixels are produced, the detector performs connected-component filtering. Each component is evaluated by length, span, thickness, density, skeleton length, elongation, texture strength, and overlap with trusted responses. Broad glare bands, smooth yolk-like curves, scattered shell pores, short fragments, and thick texture blobs are rejected. Components that satisfy crack-like geometry are kept and consolidated into a visible trace mask. The final overlay draws the trace over the original image so the user can inspect the area that influenced the decision.

The detector also returns fuzzy labels. Egg size is estimated from the apparent area, width, and length of the detected egg, while crack size is estimated from trace length, area, strength, and number of components. These labels are not physical measurements; they are apparent image classifications that depend on camera distance and image scale. Returning fuzzy memberships helps the system communicate uncertainty instead of only returning a hard label.

Increment 4: Single-Image Detection Workflow

The fourth increment connected the detection engine to a complete single-image user workflow. The detection screen allows the user to choose a supported image file. After selection, the screen reads the image bytes and sends them to DetectionCubit. The Cubit changes the interface state from idle to loading, calls the DetectionRepository, waits for the backend response, and then emits either a success state with DetectionResult or a failure state with a readable error message.

The DetectionResult model maps the backend JSON response into Dart fields used by the Flutter interface. It stores the verdict, confidence, area ratio, contour length, processing time, original image, overlay image, candidate counts, threshold used, shell texture metrics, thin crack values, egg detection score, egg size, crack size, crack mask, crack locations, and consistency values. This model acts as the contract between the backend algorithm and the frontend display.

The result interface was built to help the user understand the output. It shows the original image and overlay image, displays a cracked or not-cracked verdict badge, and lists important readouts such as detection score, confidence, egg size, crack size, and processing time. When intermediate steps are requested, the application can show processing masks and candidate maps. This makes the system more transparent because the user can see both the final answer and the visual basis of the answer.

Increment 5: Camera Capture and Focus Workflow

The fifth increment added the camera workflow. Camera handling was placed in Flutter because the backend was intentionally designed not to own cameras. `CameraSessionService` discovers available cameras, opens the selected camera, manages the preview controller, handles app lifecycle changes, and reconnects the preview when necessary. The camera screen listens to the session service so the interface can update when the camera is detecting, initializing, ready, focusing, or unavailable.

Focus support was added because crack detection requires a clear image. The camera workflow can capture focus frames and send them to the backend focus score endpoint. The backend detects the egg area and computes focus-related measurements inside the egg mask. The frontend then uses these scores to help lock or improve focus before the final capture. This prevents the user from submitting a blurred image that would later be rejected by the quality gate.

When the user captures an image, Flutter takes the picture and sends the original bytes to the camera detection endpoint. The backend applies camera-specific orientation correction and a camera detection configuration. For faster live-camera work, the detector can use a smaller target frame and a fast detection mode. For stronger validation, multi-frame detection can require a crack trace to appear in spatially matching positions across multiple frames. This reduces false detections caused by momentary glare, noise, or movement.

The camera screen presents the capture result with the same `DetectionResult` model used by the single-image workflow. The user can save the result to history when the output is acceptable or return to the camera preview to capture another image. This completes the camera increment by connecting camera discovery, focus preparation, capture, backend analysis, result display, and history saving into one workflow.

Increment 6: Batch Processing, History, Reports, and Deployment

The sixth increment added supporting features needed for practical use. Batch processing allows the user to select multiple image files or a folder of images. The application creates a queue, processes each image, updates the status of each item, and continues even if one file fails. This design is useful when many egg images must be checked because the user does not need to repeat the single-image workflow for every file.

History storage was implemented in the backend through a local JSON file. When a result is saved, the backend stores the important fields and removes heavy internal values so the history file remains usable. The store is idempotent for records with the same identifier, supports listing, retrieving, deleting one record, and clearing all records, and writes changes through an atomic replace operation. The Flutter history screen consumes these endpoints to display past detections and manage saved records.

The reports feature was built on top of the same saved history. The reports screen loads history records, summarizes detection counts and result metrics, and calls the backend export endpoint to download a CSV file. The exported file includes fields such as identifier, source name, verdict, egg size, crack size, quality metrics, confidence, area ratio, contour length, processing time, and timestamp. This allows the detection results to be attached to capstone documentation or opened in spreadsheet software for review.

Deployment support was added through backend scripts and launcher logic. The backend includes run scripts for development and build scripts for packaging the Python backend as an executable. The Flutter application uses the backend launcher so the local API can be started automatically from the desktop app. This makes the system closer to an installable local tool instead of a prototype that requires the user to manually run several commands.

Testing Phase

Testing was conducted after each increment and again after integration. Backend tests verify the detector and API contract. The detector tests include clean eggs, dark cracks, bright cracks, faint cracks, textured shells, glare, blur, low contrast, camera focus scoring, multi-frame trace validation, and fuzzy egg-size membership. These tests were necessary because the detection algorithm contains many thresholds and filters that must work together.

API contract tests verify that the backend exposes the expected routes and keeps camera ownership in the frontend. They also check that camera detection endpoints are named correctly, camera image orientation is preserved, fuzzy egg-size memberships are normalized, and history saving is idempotent. These tests protect the communication contract between

Flutter and the backend.

Frontend testing included widget-level startup checking and manual workflow testing of the desktop screens. Manual testing was important for camera preview, file picking, backend startup, result rendering, history management, and CSV saving because these functions interact with local files, hardware, and the operating system. Errors returned from the backend are converted into user-friendly messages so the operator knows whether to recapture, select a different file, or restart the detection service.

Deployment Phase

The deployment phase prepared the system for local desktop execution. The Flutter application runs as the user-facing desktop program, while the backend runs locally on the configured address. The backend provides a `/health` endpoint so the frontend can check readiness before sending requests. This health check is used during startup and before detection calls to avoid requesting analysis from a backend that is not yet available.

Backend deployment support includes scripts for running the server, installing requirements, building an executable, copying the backend to the desktop app, and starting it silently when needed. The frontend deployment includes assets, icons, sample egg images, and configuration constants for API paths and timeouts. Together, these deployment decisions allow the system to behave like one local application even though it is internally composed of a Flutter frontend and Python backend.

Maintenance Phase

Maintenance in the incremental model continues after deployment. For this system, maintenance mainly involves improving detection thresholds, adding more test images, refining error messages, updating camera handling for different hardware, and expanding reports. Because the system is modular, the detector can be improved in the backend without redesigning every Flutter screen. Likewise, the user interface can be improved without changing the OpenCV pipeline.

Future increments may include calibration for physical egg-size measurement, database-backed history storage, role-based user accounts, more export formats, and improved camera calibration. These changes can be added as new increments because the current design already separates frontend workflow, backend endpoints, detection logic, and stored detection records.

Chapter Summary

The methodology used for the Egg Crack Detection System followed the incremental model. The system was planned, analyzed, designed, implemented, tested, deployed, and prepared for maintenance through a series of working increments. This approach matched the nature of the project because the desktop interface, API service, image-processing engine, camera workflow, history storage, batch processing, and report generation could be developed and validated one at a time. The resulting system is a local Flutter and FastAPI application that accepts egg images, processes them through OpenCV, displays visual detection results, stores accepted records, and exports detection reports.